MANCHESTER 1824
The University of Manchester

Machine Learning Lab

EEEN60151 & EEEN40151

Department of Electrical and Electronic Engineering

December 2025

Student ID: 11049301

Name: Qais Momani

Contents

Contents 2
List of Figures 2
List of Tables 3
1 Introduction 4
2 Task 1: Single Perceptron 4
2.1 Methodology 4
2.2 Results 4
2.3 Discussion 6
3 Task 2: Single Layer Perceptrons 6
3.1 Methodology 6
3.2 Results 7
3.3 Discussion 9
4 Task 3: Multi-Layer Perceptron 9
4.1 Methodology 9
4.2 Results 9
4.3 Discussion 11
5 Conclusion 11
Appendices 12

List of Figures

Figure 1- Learning curve showing the total classification error reducing to zero over training epochs. 5
Figure 2- MATLAB Command Window results showing the independent test results, and a summary of the average test accuracy and epochs taken to converge. 5

Figure 3- Colour visualisation of learned weights, generated by MATLAB. 6
Figure 4- Learning curve for the SLP on MNIST. 7
Figure 5- Visualization of the weight matrices for all 10 digits. 8
Figure 6- Confusion matrix showing the distribution of predicted vs. true labels. 8
Figure 7- Learning curve for the MLP (20 hidden nodes). 10
Figure 8- Confusion matrix for the MLP. 10

List of Tables

Table 1- Performance comparison of MLPs with different hidden layer capacities. 10

1 Introduction

This report investigates the relationship between neural network complexity and classification performance in handwritten digit recognition. Using the MNIST dataset, the study progresses from a fundamental Single Perceptron for binary classification to a Single Layer Perceptron (SLP) for multi-class tasks, and finally to a Multilayer Perceptron (MLP) employing backpropagation. By evaluating these architectures sequentially, this report demonstrates the necessity of non-linear decision boundaries for complex pattern recognition and analyses the trade-offs between computational efficiency and the model's ability to resolve geometrically similar digits.

2 Task 1: Single Perceptron

2.1 Methodology

To simulate a simplified digit recognition problem, a dataset was manually constructed consisting of 12 binary images comprised of 5x5 pixels. These represented two classes: the digit "0" and the digit "1", with 6 distinct variations per class to capture handwriting diversity. Each 5x5 matrix was flattened into a 25-dimensional column vector, serving as the input to the network.

A single Rosenblatt Perceptron was implemented. The output *y* is determined by the sign of the weighted sum of inputs plus a bias as shown in equation 1:

$$y = sign(w^T x + b) \tag{1}$$

Where the weights, *w*, and bias, *b,* were initialised with small random numbers. The target labels, d, were set to 1 for class “1” and -1 for class “0”.

The network was trained using the standard Perceptron Learning Rule. For each training sample where the prediction was incorrect $(y \neq d)$, the weights and bias were updated as shown by equations 2 and 3:

$$w_{new} = w_{old} + \alpha(d - y)x \tag{2}$$

$$b_{new} = b_{old} + \alpha(d - y) \tag{3}$$

A learning rate α = 0.1 was used.

To evaluate the robustness of the training algorithm, the experiment was repeated for 10 independent trials. In each trial, the dataset was randomly partitioned into a Training Set consisting of 4 samples per class and a Test Set consisting of 2 samples per class.

2.2 Results

The perceptron demonstrated rapid convergence in all trials. Figure 1 illustrates the training error over epochs for a typical trial. The error consistently dropped to zero within the first few epochs with an average of 2.20 epochs, indicating the training set was perfectly classified.

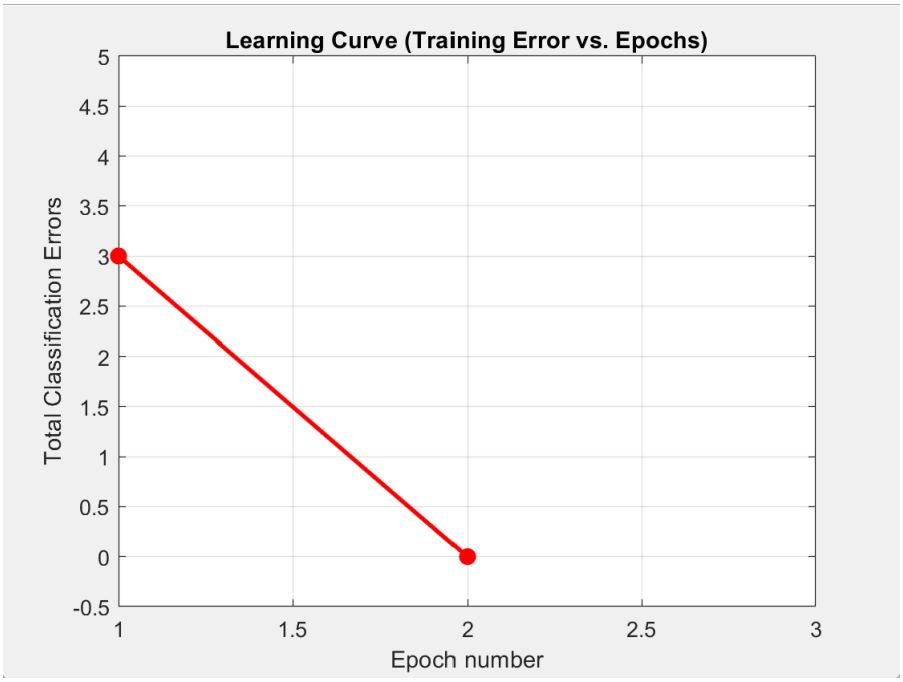


Figure 1- Learning curve showing the total classification error reducing to zero over training epochs.

Figure 2 shows the MATLAB command window after running the simulation, which summarises the results across the 10 independent runs. The model achieved an average test accuracy of 85%, with several runs showcasing a 100% accuracy.

```
Trial 1: Converged in 2 epochs | Test Accuracy: 100.0%
Trial 2: Converged in 2 epochs | Test Accuracy: 75.0%
Trial 3: Converged in 2 epochs | Test Accuracy: 75.0%
Trial 4: Converged in 3 epochs | Test Accuracy: 100.0%
Trial 5: Converged in 3 epochs | Test Accuracy: 100.0%
Trial 6: Converged in 2 epochs | Test Accuracy: 75.0%
Trial 7: Converged in 2 epochs | Test Accuracy: 100.0%
Trial 8: Converged in 2 epochs | Test Accuracy: 100.0%
Trial 9: Converged in 2 epochs | Test Accuracy: 50.0%
Trial 10: Converged in 2 epochs | Test Accuracy: 75.0%

------------------------------------------------
RESULTS SUMMARY (Over 10 Trials)
------------------------------------------------
Average Test Accuracy: 85.00%
Average Epochs to Converge: 2.20
------------------------------------------------
```

Figure 2- MATLAB Command Window results showing the independent test results, and a summary of the average test accuracy and epochs taken to converge.

To interpret what the perceptron learned, the final 25-dimensional weight vector was reshaped into a 5x5 grid, shown in Figure 3.

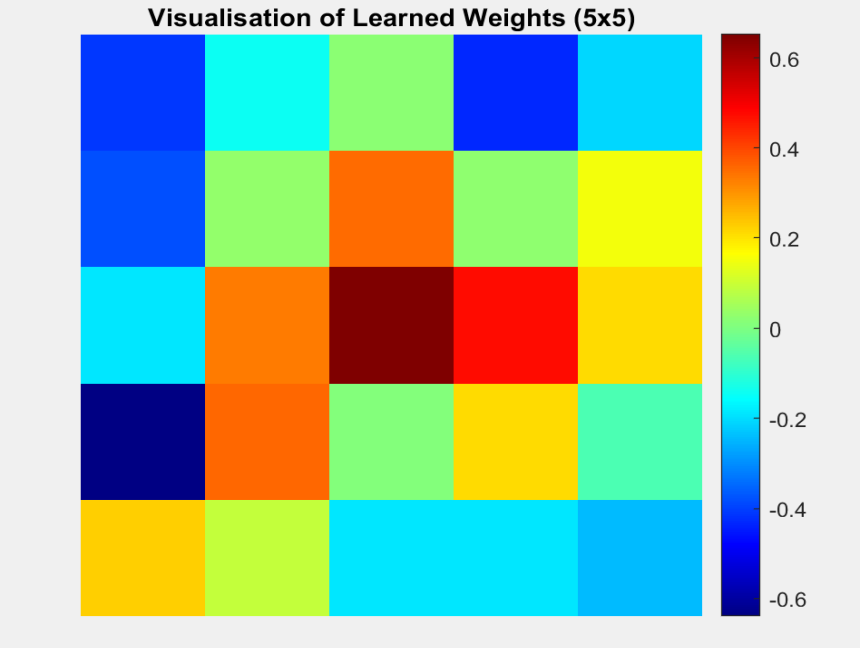


Figure 3- Colour visualisation of learned weights, generated by MATLAB.

The red pixels with values greater than 0 represent positive weights for class “1”, and the blue pixels less than 0 represent negative weights for class “0”.

2.3 Discussion

The Perceptron Convergence Theorem states that if two classes are linearly separable, the perceptron algorithm is guaranteed to find a solution in finite time. The rapid convergence observed in Figure 1 confirms that the constructed 5x5 patterns for "0" and "1" are indeed linearly separable. Figure 3 offers insight into the model's decision-making. The weights formed a template:

Positive weights (Red): Clustered in the centre vertical column, corresponding to the typical shape of the digit "1".

Negative weights (Blue): Clustered around the top, bottom, and side edges where the digit "0" typically has pixels but "1" does not.

This demonstrates that the perceptron effectively learned a spatial filter to discriminate the classes. While training error always reached zero, test accuracy varied (averaging 85%). This discrepancy is due to the Small Sample Size problem. With only 4 training samples per class, the model sometimes overfitted to specific variations and failed to generalize to unseen test variations. This limitation motivates the use of larger dataset in the following tasks.

3 Task 2: Single Layer Perceptrons

3.1 Methodology

This task utilised the MNIST dataset, a standard benchmark containing 60,000 training images and 10,000 testing images of handwritten digits (0-9). Each image is a 28x28

grayscale grid, flattened into a 784-dimensional vector *x*. Pixel values were normalised to the range [0, 1] by dividing by 255. The integer labels were converted into One-Hot Encoded vectors $d \in \{0,1\}^{10}$. For example, the digit "3" is represented as $[0,0,0,1,0,0,0,0,0,0]^T$. The Single Layer Perceptron (SLP) network consisted of 10 parallel perceptrons, one for each digit class, connected directly to the 784 inputs. Unlike the binary step function used in Task 1, this model employed the Sigmoid Activation Function, equation 4, to provide a continuous, differentiable output between 0 and 1:

$$y_j = \frac{1}{1+e^{-vj}} \; where \; v_j = w_j^T x + b_j \tag{4}$$

The network was trained using Gradient Descent via the Delta Rule. The objective was to minimise the Sum of Squared Errors (SSE) between the predicted output vector, *y,* and the target vector, *d*. The weight update rule, derived from the derivative of the sigmoid function, is:

$$\Delta w_{ji} = \alpha(d_j - y_j)y_j(1-y_j)x_i \tag{5}$$

A learning rate of α = 0.1 was used over 20 epochs. Classification was performed by selecting the neuron with the maximum output value: $Class = arg\ max_j(y_j)$.

3.2 Results

The SLP demonstrated strong baseline performance, significantly outperforming random guessing but plateauing below perfect accuracy.

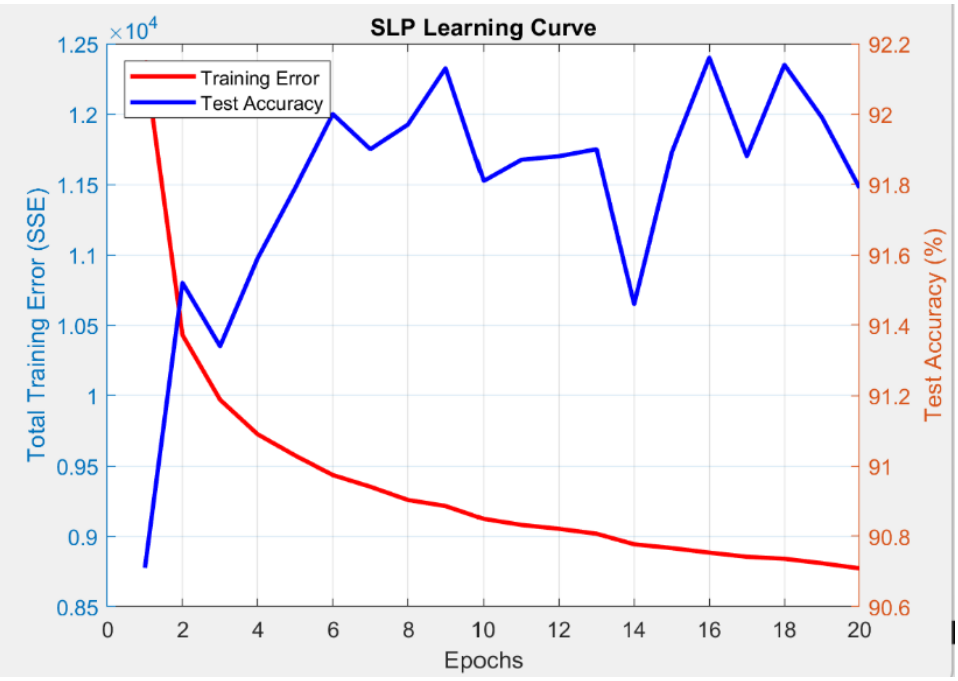


Figure 4- Learning curve for the SLP on MNIST.

Figure 4 shows the training progression. The Total Training Error (red line) decreased smoothly, indicating successful gradient descent. The Test Accuracy (blue line) rose sharply in the first 5 epochs and stabilized at approximately 92%.

Figure 5 displays the learned weights for all 10 perceptrons reshaped into 28x28 images. The weights resemble templates of the digits they are trained to recognize. For instance, the weights for digit "0" roughly show a ring of high values (red) surrounding a centre of low values (blue), acting as a matched filter.

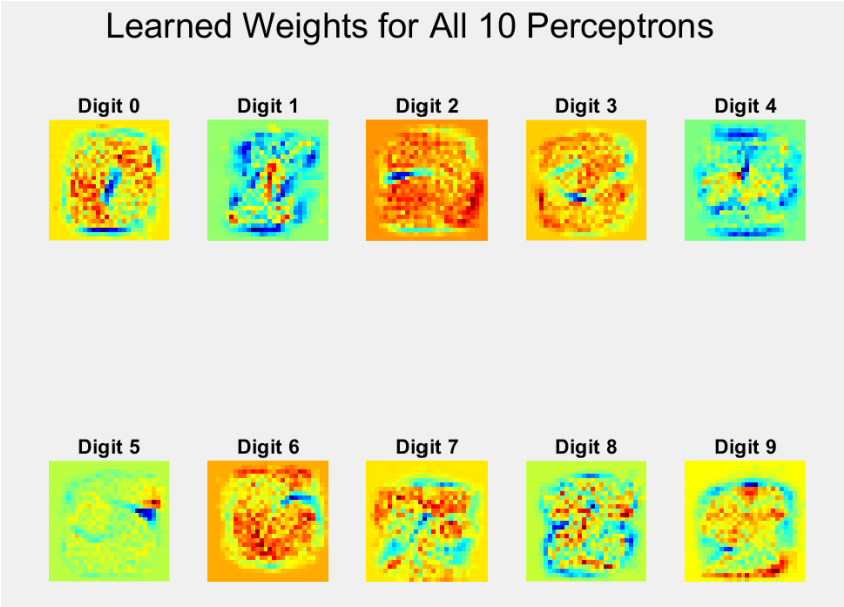


Figure 5- Visualization of the weight matrices for all 10 digits.

To analyse misclassifications, a confusion matrix was generated as seen in Figure 6. While the diagonal elements are dominant (correct classifications), specific off-diagonal clusters reveal systematic errors. The most frequent errors occurred between geometrically similar digits, specifically ‘4’ vs ‘9’, ‘3’ vs ‘5’, and ‘7’ vs ‘9’.

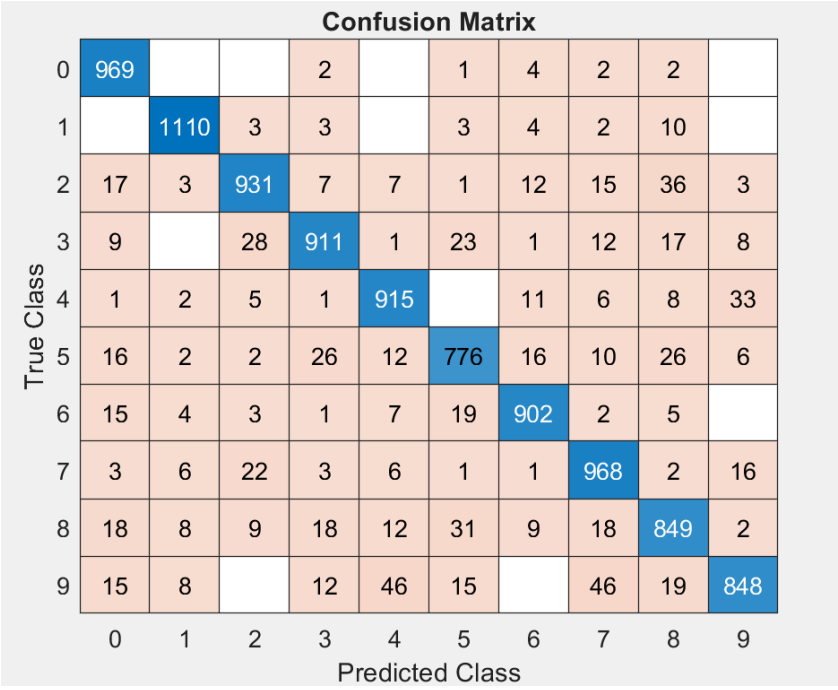


Figure 6- Confusion matrix showing the distribution of predicted vs. true labels.

3.3 Discussion

The visualisation in Figure 5 confirms that the SLP functions as a linear template matcher. It learns an average representation of each digit. It performs well when a test digit aligns with this average template but fails when the handwriting style deviates significantly, such as a slanted ‘1’ or a closed-top ‘4’.

The accuracy plateau at ~92% highlights the fundamental limitation of Single Layer Perceptrons: they can only classify data that is linearly separable. The MNIST digits are complex patterns that overlap in the 784-dimensional pixel space.

The confusion between ‘4’ and ‘9’ is a direct result of linearity. As ‘4’ and a ‘9’ share many pixel locations (the vertical stem and the upper loop), a linear classifier sums these pixel contributions independently and struggles to learn the non-linear relationship required to distinguish them reliably. If the top is open, it is a 4; if closed, it is a 9. This limitation motivates the need for hidden layers in Task 3.

4 Task 3: Multi-Layer Perceptron

4.1 Methodology

To overcome the linear limitations of the SLP, a Multilayer Perceptron (MLP) was implemented with a single hidden layer. The architecture consists of:

An Input Layer of 784 neurons (normalised pixel values), a Hidden Layer of *K* neurons with Sigmoid activation, and an Output Layer of 10 neurons (one per digit) with Sigmoid activation.

The signal propagates through the network via the two non-linear transformations, equations 6 and 7:

$$y_{hidden} = \varphi(W_h x + b_h) \tag{6}$$

$$y_{output} = \varphi(W_o y_{hidden} + b_o) \tag{7}$$

The network was trained using the generalised Delta Rule. This involves a two-step gradient calculation, calculating the Output Error in equation 8 and the Hidden Error in equation 9:

$$\delta_o = (d - y_o) y_o (1 - y_o) \tag{8}$$

$$\delta_h = (W_o^T \delta_o) y_h (1 - y_h) \tag{9}$$

To investigate the effect of model capacity, the network was trained with three different hidden layer sizes: $K = 10$, $K = 20$, $K = 50$.

4.2 Results

The introduction of the hidden layer resulted in a distinct improvement in classification accuracy compared to the single-layer models. Table 1 summarises the performance across the three configurations. Increasing the number of hidden nodes generally improved test accuracy. The configuration with $K = 50$ achieved the highest performance, though the gain over $K = 20$ was marginal compared to the 116.73s increase in training time.

Table 1- Performance comparison of MLPs with different hidden layer capacities.

No. of Hidden Nodes	Final Test Accuracy	Epoch Number	Elapsed Time
10	92.30%	20	55.45s
20	95.26%	20	74.25s
50	97.04%	20	190.98s

Figure 7 illustrates the learning curve for the K = 20 model. Compared to the SLP, the training error drops lower , and the test accuracy climbs higher to approximately 95%, indicating the model is learning a more precise fit to the data.

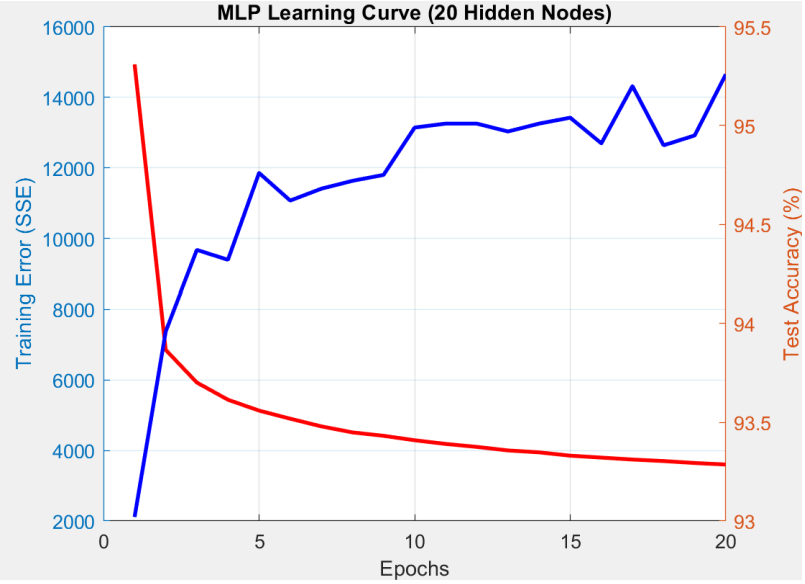


Figure 7- Learning curve for the MLP (20 hidden nodes).

Figure 8 displays the confusion matrix for the MLP K = 20.

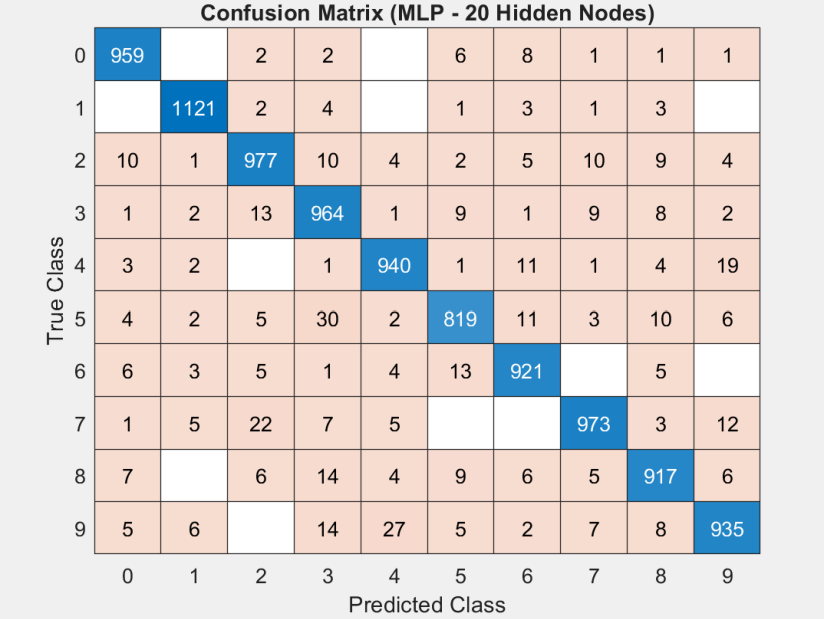


Figure 8- Confusion matrix for the MLP.

The systematic errors observed in Task 2 were significantly reduced.

The confusion between '4' and '9' dropped from 33 and 46 errors in the SLP to 27 and 19 errors. This confirms the MLP's ability to distinguish subtle geometric differences that misled the linear classifier.

4.3 Discussion

The performance jump from ~92% (SLP) to ~95%+ (MLP) demonstrates the necessity of non-linear decision boundaries for handwritten digit recognition. The hidden neurons function as feature detectors; they learn to identify specific sub-patterns rather than just matching a global template. The output layer then combines these non-linear features to form a more robust classification.

Capacity and Overfitting:

- **K=10:** While better than an SLP, 10 nodes may create a bottleneck, forcing the network to compress 784 pixels into too few features, potentially losing information.
- **K=50:** Provided the best accuracy but required more computational resources. With significantly more parameters, larger networks risk overfitting, though the results here suggest the regularising effect of the limited epoch count prevented this.

The results suggest that $K = 20$ represents an optimal trade-off for this specific dataset, providing sufficiently high accuracy (within 1% of the larger model) while maintaining faster training speeds than the $K = 50$ configuration.

5 Conclusion

This investigation successfully validated that while simple linear models are computationally efficient, they possess fundamental limitations when applied to complex, high-dimensional data like MNIST.

- **Linear Limitations:** The Single Perceptron and SLP excelled at tasks with linear separability, achieving rapid convergence on binary data. However, the SLP hit a performance ceiling of approximately 92% on the full dataset, consistently failing to distinguish overlapping classes such as ‘4’ and ‘9’.
- **Non-Linear Advantage:** The implementation of the MLP and backpropagation effectively overcame this barrier. The hidden layer acted as a feature extractor, learning non-linear representations that resolved geometric ambiguities, thereby boosting accuracy to over 97% with 50 hidden layers.
- **Trade-off:** The experiment confirms that the increased computational cost of training hidden layers is a necessary investment for robust generalisation. Future work to further improve performance would require deep learning architectures, such as Convolutional Neural Networks (CNNs), to explicitly exploit spatial correlations that fully connected MLPs ignore.

Appendices

Appendix A: Task 1 Code

```
% Class '1' Patterns (Target = 1)
p1_1 = [0 0 1 0 0; 0 0 1 0 0; 0 0 1 0 0; 0 0 1 0 0; 0 0 1 0 0];
p1_2 = [0 1 1 0 0; 0 0 1 0 0; 0 0 1 0 0; 0 0 1 0 0; 0 1 1 1 0];
p1_3 = [0 1 1 0 0; 0 0 1 0 0; 0 0 1 0 0; 0 0 1 0 0; 0 0 1 0 0];
p1_4 = [0 0 0 1 0; 0 0 1 1 0; 0 1 1 0 0; 1 1 0 0 0; 1 0 0 0 0];
p1_5 = [0 0 0 0 1; 0 0 0 1 1; 0 0 1 1 0; 0 1 1 0 0; 1 1 0 0 0];
p1_6 = [0 0 1 0 0; 0 0 1 0 0; 0 0 1 0 0; 0 0 1 0 0; 0 0 1 1 0];

% Class '0' Patterns (Target = -1)
p0_1 = [1 1 1 1 1; 1 0 0 0 1; 1 0 0 0 1; 1 0 0 0 1; 1 1 1 1 1];
p0_2 = [0 1 1 1 0; 1 0 0 0 1; 1 0 0 0 1; 1 0 0 0 1; 0 1 1 1 0];
p0_3 = [0 1 1 0 0; 1 0 0 1 0; 1 0 0 1 0; 1 0 0 1 0; 0 1 1 0 0];
p0_4 = [0 0 1 1 0; 0 1 0 0 1; 0 1 0 0 1; 0 1 0 0 1; 0 0 1 1 0];
p0_5 = [1 1 1 1 0; 1 0 0 1 0; 1 0 0 1 0; 1 0 0 1 0; 1 1 1 1 0];
p0_6 = [0 1 1 1 1; 1 0 0 0 1; 1 0 0 0 1; 1 0 0 0 1; 0 1 1 1 1];

% Flatten to column vectors (25x1) and store in cell array

data_1 = {p1_1(:), p1_2(:), p1_3(:), p1_4(:), p1_5(:), p1_6(:)};
data_0 = {p0_1(:), p0_2(:), p0_3(:), p0_4(:), p0_5(:), p0_6(:)};


n_trials = 10;          % Repeat 10 times to get average performance
max_epochs = 50;        % Max training loops
learning_rate = 0.1;    % Alpha
input_dim = 25;         % Dimension of input vector

% Storage for results
trial_accuracies = zeros(n_trials, 1);
trial_epochs = zeros(n_trials, 1);
errors_last_trial = []; % To store error curve of the last trial for plotting

fprintf('Starting %d Simulation Trials...\n\n', n_trials);


for trial = 1:n_trials

    % Randomly Split Data (4 Train / 2 Test per class)
    perm_1 = randperm(6);
    perm_0 = randperm(6);

    % Training Set
    train_idx_1 = perm_1(1:4);
    train_idx_0 = perm_0(1:4);

    % Test Set
    test_idx_1 = perm_1(5:6);
    test_idx_0 = perm_0(5:6);

    % Construct Training Matrix X and Label Vector D

    X_train = [data_1{train_idx_1}, data_0{train_idx_0}]; % 25x8 matrix
    D_train = [ones(1, 4), -ones(1, 4)];                  % 1x8 vector
```

```
% Construct Test Matrix
X_test = [data_1{test_idx_1}, data_0{test_idx_0}];    % 25x4 matrix
D_test = [ones(1, 2), -ones(1, 2)];                   % 1x4 vector

% Initialize Weights and Bias ---
w = rand(input_dim, 1) - 0.5; % Small random weights
b = rand() - 0.5;             % Small random bias

% Training Loop (Perceptron Learning Rule)
epoch_errors = [];

for epoch = 1:max_epochs
    total_error = 0;

    % Randomise presentation order within epoch
    perm_train = randperm(8);

    for k = perm_train
        x_k = X_train(:, k);
        d_k = D_train(k);

        % Calculate Output: y = sign(w'*x + b)
        v = w' * x_k + b;
        y_k = sign(v);
        if y_k == 0, y_k = 1; end % Handle sign(0) case if needed

        % Update Weights if Error Exists
        if y_k ~= d_k
            % Update Rules
            w = w + learning_rate * (d_k - y_k) * x_k;
            b = b + learning_rate * (d_k - y_k);
            total_error = total_error + 1;
        end
    end

    epoch_errors = [epoch_errors, total_error];

    % Stop if converged
    if total_error == 0
        break;
    end
end

trial_epochs(trial) = length(epoch_errors);

% Testing
% Apply trained network to test patterns
n_correct = 0;
n_test_samples = size(X_test, 2);

for k = 1:n_test_samples
    y_test = sign(w' * X_test(:, k) + b);
    if y_test == 0, y_test = 1; end

    if y_test == D_test(k)
        n_correct = n_correct + 1;
    end
end

% Calculate Percentage Error Rate
```

```
    accuracy = (n_correct / n_test_samples) * 100;
    trial_accuracies(trial) = accuracy;


    if trial == n_trials
        errors_last_trial = epoch_errors;
    end

    fprintf('Trial %d: Converged in %d epochs | Test Accuracy: %.1f%%\n', ...
        trial, trial_epochs(trial), accuracy);
end



% Calculate Averages
avg_accuracy = mean(trial_accuracies);
avg_epochs = mean(trial_epochs);

fprintf('\n------------------------------------------------\n');
fprintf('RESULTS SUMMARY (Over %d Trials)\n', n_trials);
fprintf('------------------------------------------------\n');
fprintf('Average Test Accuracy: %.2f%%\n', avg_accuracy);
fprintf('Average Epochs to Converge: %.2f\n', avg_epochs);
fprintf('------------------------------------------------\n');

% Plot Learning Curve
figure;
plot(1:length(errors_last_trial), errors_last_trial, '-ro', 'LineWidth', 2,
'MarkerFaceColor', 'r');
title('Learning Curve (Training Error vs. Epochs)');
xlabel('Epoch number');
ylabel('Total Classification Errors');
grid on;
axis([1 max(trial_epochs) -0.5 5]);

% Visualize the learned weights
% Reshape 25x1 weights back to 5x5 to see which pixels matter most
figure;
imagesc(reshape(w, 5, 5));
colormap('jet');
colorbar;
title('Visualisation of Learned Weights (5x5)');
axis square off;
```

Appendix B: Task 2 Code

```
fprintf('Loading MNIST Data...\n');
if exist('mnist.mat', 'file')
    d = load('mnist.mat');
else
    error('Error: mnist.mat not found. Please ensure it is in the current folder.');
end

% Extract and Preprocess Training Data
% Transpose to make them column vectors
X_train = double(d.trainX') / 255.0;  % Normalise to [0, 1]
Y_train_labels = double(d.trainY);

% Extract and Preprocess Test Data
X_test = double(d.testX') / 255.0;    % Normalise to [0, 1]
Y_test_labels = double(d.testY);



[input_dim, num_train] = size(X_train);
num_test = size(X_test, 2);
num_classes = 10; % Digits 0-9

% Convert Labels to One-Hot Target Vectors (10 x N)

D_train = zeros(num_classes, num_train);
for i = 1:num_train

    D_train(Y_train_labels(i) + 1, i) = 1;
end



W = rand(num_classes, input_dim) * 0.01 - 0.005; % Small random weights
b = rand(num_classes, 1) * 0.01 - 0.005;         % Small random bias

learning_rate = 0.1;
max_epochs = 20; % Maximum Epochs

% Storage for plotting
train_error_history = [];
test_accuracy_history = [];

fprintf('Starting Training on %d samples...\n', num_train);


for epoch = 1:max_epochs

    total_epoch_error = 0;

    % Shuffle training data each epoch
    perm = randperm(num_train);

    for k = perm
        x_k = X_train(:, k); % Current image (784x1)
        d_k = D_train(:, k); % Target vector (10x1)
```

```
        % Calculate weighted sum: v = Wx + b
        v = W * x_k + b;

        % Sigmoid Activation: y = 1 / (1 + e^-v)
        y_k = 1 ./ (1 + exp(-v));


        % Error signal: e = d - y
        e = d_k - y_k;

        % Calculate local gradient delta

        delta = e .* y_k .* (1 - y_k);

        % Update Weights and Biases
        % W_new = W + alpha * delta * x'
        W = W + learning_rate * (delta * x_k');
        b = b + learning_rate * delta;

        % Sum of Squared Errors
        total_epoch_error = total_epoch_error + sum(e.^2);
    end

    % Store Error
    train_error_history = [train_error_history, total_epoch_error];

    % Evaluate on Test Set every epoch
    correct_count = 0;
    for t = 1:num_test
        x_t = X_test(:, t);

        % Forward pass
        v_t = W * x_t + b;
        y_t = 1 ./ (1 + exp(-v_t));

        % Classification: Max output index
        [~, predicted_digit_idx] = max(y_t);
        predicted_digit = predicted_digit_idx - 1;

        if predicted_digit == Y_test_labels(t)
            correct_count = correct_count + 1;
        end
    end
    test_acc = (correct_count / num_test) * 100;
    test_accuracy_history = [test_accuracy_history, test_acc];

    fprintf('Epoch %d | Error: %.2f | Test Accuracy: %.2f%%\n', ...
        epoch, total_epoch_error, test_acc);
end



% Figure 1: Learning Curve
figure;
yyaxis left
plot(1:max_epochs, train_error_history, '-r', 'LineWidth', 2);
ylabel('Total Training Error (SSE)');
xlabel('Epochs');

yyaxis right
```

```
plot(1:max_epochs, test_accuracy_history, '-b', 'LineWidth', 2);
ylabel('Test Accuracy (%)');
title('Task 2: SLP Learning Curve');
legend('Training Error', 'Test Accuracy');
grid on;

% Figure 2: Visualising Learned Weights

figure;
for i = 1:10
    subplot(2, 5, i);
    % Reshape the row of weights for digit (i-1)
    weight_img = reshape(W(i, :), 28, 28);
    imagesc(weight_img');
    colormap('jet');
    axis square off;
    title(['Digit ' num2str(i-1)]);
end
sgtitle('Learned Weights for All 10 Perceptrons');

% Figure 3: Confusion Matrix
% Generate predictions for the whole test set for the matrix
all_preds = zeros(1, num_test);
for t = 1:num_test
    y_t = 1 ./ (1 + exp(-(W * X_test(:, t) + b)));
    [~, idx] = max(y_t);
    all_preds(t) = idx - 1;
end

figure;
confusionchart(Y_test_labels, all_preds);
title('Confusion Matrix (Task 2)');
```

Appendix C: Task 3 Code

```
tic

fprintf('Loading MNIST Data...\n');
if exist('mnist.mat', 'file')
    d = load('mnist.mat');
else
    error('Error: mnist.mat not found.');
end

% Normalise Data
X_train = double(d.trainX') / 255.0;
Y_train_labels = double(d.trainY);
X_test = double(d.testX') / 255.0;
Y_test_labels = double(d.testY);

[input_dim, num_train] = size(X_train);
num_test = size(X_test, 2);
num_classes = 10;

% One-Hot Encoding for Targets
D_train = zeros(num_classes, num_train);
for i = 1:num_train
    D_train(Y_train_labels(i) + 1, i) = 1;
end


num_hidden = 20;  % Number of hidden neurons
learning_rate = 0.1;
max_epochs = 20;

% Initialise Weights
% Input -> Hidden Weights (W_h) and Biases (b_h)
W_h = rand(num_hidden, input_dim) * 0.02 - 0.01;
b_h = rand(num_hidden, 1) * 0.02 - 0.01;

% Hidden -> Output Weights (W_o) and Biases (b_o)
W_o = rand(num_classes, num_hidden) * 0.02 - 0.01;
b_o = rand(num_classes, 1) * 0.02 - 0.01;

% Storage for plotting
train_error_history = [];
test_accuracy_history = [];

fprintf('Starting MLP Training (Hidden Nodes: %d)...\n', num_hidden);


for epoch = 1:max_epochs
    total_epoch_error = 0;
    perm = randperm(num_train);

    for k = perm
        x_k = X_train(:, k);
        d_k = D_train(:, k);


        % 1. Input to Hidden
        v_h = W_h * x_k + b_h;
```

```
        y_h = 1 ./ (1 + exp(-v_h)); % Sigmoid activation

        % 2. Hidden to Output
        v_o = W_o * y_h + b_o;
        y_o = 1 ./ (1 + exp(-v_o)); % Sigmoid activation



        % Calculate Output Error and Delta
        e_o = d_k - y_o;
        % delta_o = error * derivative of sigmoid
        delta_o = e_o .* y_o .* (1 - y_o);

        % Calculate Hidden Layer Delta

        e_h = W_o' * delta_o;
        delta_h = e_h .* y_h .* (1 - y_h);

        % Update Weights and Biases
        % Update Hidden->Output
        W_o = W_o + learning_rate * (delta_o * y_h');
        b_o = b_o + learning_rate * delta_o;

        % Update Input->Hidden
        W_h = W_h + learning_rate * (delta_h * x_k');
        b_h = b_h + learning_rate * delta_h;

        total_epoch_error = total_epoch_error + sum(e_o.^2);
    end


    correct_count = 0;
    for t = 1:num_test
        x_t = X_test(:, t);
        % Forward pass only
        y_h_test = 1 ./ (1 + exp(-(W_h * x_t + b_h)));
        y_o_test = 1 ./ (1 + exp(-(W_o * y_h_test + b_o)));

        [~, pred] = max(y_o_test);
        if (pred - 1) == Y_test_labels(t)
            correct_count = correct_count + 1;
        end
    end
    test_acc = (correct_count / num_test) * 100;

    train_error_history = [train_error_history, total_epoch_error];
    test_accuracy_history = [test_accuracy_history, test_acc];

    fprintf('Epoch %d | Error: %.2f | Test Acc: %.2f%%\n', epoch, total_epoch_error,
test_acc);
end



% Plot Learning Curve
figure;
yyaxis left
plot(1:max_epochs, train_error_history, '-r', 'LineWidth', 2);
ylabel('Training Error (SSE)');
xlabel('Epochs');
```

```
yyaxis right
plot(1:max_epochs, test_accuracy_history, '-b', 'LineWidth', 2);
ylabel('Test Accuracy (%)');
title(['Task 3: MLP Learning Curve (' num2str(num_hidden) ' Hidden Nodes)']);
grid on;

% Confusion Matrix for MLP
all_preds = zeros(1, num_test);
for t = 1:num_test
    x_t = X_test(:, t);
    y_h_test = 1 ./ (1 + exp(-(W_h * x_t + b_h)));
    y_o_test = 1 ./ (1 + exp(-(W_o * y_h_test + b_o)));
    [~, idx] = max(y_o_test);
    all_preds(t) = idx - 1;
end

figure;
confusionchart(Y_test_labels, all_preds);
title(['Confusion Matrix (MLP - ' num2str(num_hidden) ' Hidden Nodes)']);

toc
```